

Tag 2 - Cleverere Prüfziffern und der Sprung zum Korrigieren

Gestern haben wir Fehler **erkannt**. Heute wagen wir den größeren Schritt: Wir wollen sie **reparieren**. Dazu lernen wir zuerst zwei Verfahren aus dem echten Leben kennen (ISBN-10, Luhn) und stoßen dabei auf den zentralen Begriff der Fehlerkorrektur überhaupt: die **Hamming-Distanz**.

Lernziele für heute

Am Ende von Tag 2 kannst du ...

- die ISBN-10-Prüfziffer berechnen und erklären, warum sie Zifferndreher *immer* erkennt
- mit dem Luhn-Algorithmus eine Kreditkartennummer prüfen
- den Unterschied zwischen *Integritätsprüfung* und *Sicherheit* erklären
- die **Hamming-Distanz** zwischen zwei Wörtern berechnen
- aus der Hamming-Distanz ableiten, wie viele Fehler ein Code erkennen oder korrigieren kann
- erklären, wieso der Wiederholungscode (000, 111) 1 Fehler korrigieren kann

Material: Bleistift, kariertes Papier, Thonny. Optional: ein altes Buch mit ISBN-10 (10-stellig, vor 2007 gedruckt).

Block 1 · ISBN-10 - Wenn 10 Symbole nicht reichen (≈ 45 min)

Bis 2007 hatten Bücher eine 10-stellige **ISBN-10**. (Heute haben sie eine ISBN-13, die nach genau dem EAN-13-Schema von gestern funktioniert.) Die ISBN-10 ist mathematisch deutlich raffinierter – und sie kann etwas, was EAN-13 nicht kann.

So wird die Prüfziffer berechnet

Bei einer ISBN-10 sind die ersten 9 Ziffern Daten, die 10. ist die Prüfziffer.

1. Multipliziere die Ziffern an Position 1, 2, 3, ..., 9 mit den Gewichten **10, 9, 8, ..., 2**.
2. Bilde die Summe.
3. **Prüfziffer = (11 – (Summe mod 11)) mod 11**

Bei EAN-13 war es Modulo **10**. Hier ist es Modulo **11**. Das ist der entscheidende Unterschied – und er hat eine merkwürdige Konsequenz.

Beispiel: 3 - 4 4 6 - 4 3 5 0 0 - ?

$$\begin{aligned}
& 3 \cdot 10 + 4 \cdot 9 + 4 \cdot 8 + 6 \cdot 7 + 4 \cdot 6 + 3 \cdot 5 + 5 \cdot 4 + 0 \cdot 3 + 0 \cdot 2 \\
&= 30 + 36 + 32 + 42 + 24 + 15 + 20 + 0 + 0 \\
&= 199
\end{aligned}$$

$$\begin{aligned}
199 \bmod 11 &= 1 && (\text{denn } 199 = 18 \cdot 11 + 1) \\
11 - 1 &= 10
\end{aligned}$$

Hoppla – die Prüfziffer ist **10**. Aber 10 ist keine einzelne Ziffer! Was tun?

Lösung: Man schreibt ein **X** (römisch zehn). Vollständige ISBN-10: 3-446-43500-X.

Damit hat die ISBN-10 plötzlich **elf mögliche Symbole** für die Prüfziffer (0–9 und X), obwohl die Daten selbst nur aus zehn Ziffern bestehen. Das fühlt sich seltsam an – ist aber nötig, damit die Mathematik aufgeht. Merk dir das! In ein paar Tagen, wenn wir bei Reed-Solomon ankommen, sehen wir genau dasselbe Phänomen in groß: Codes brauchen oft mehr Symbole als die Daten.



Bleistift-
Übung 1

ISBN-10 nachrechnen

- (a) Hol dir ein altes Buch mit ISBN-10 vom Regal (oder nimm eine aus dem Internet). Decke die letzte Stelle ab und rechne sie aus.
- (b) Berechne die Prüfziffer für die Datenziffern 0306406152. (Das ist eine berühmte ISBN – kennst du das Buch? Frag nach dem Praktikum mal Google. Das 61 im siebten Kapitel ist Pflichtlektüre für jede Informatikerin.)
- (c) Berechne die Prüfziffer für 097522980. Was ist die Prüfziffer?

Warum ist Modulo 11 schlauer als Modulo 10?

Erinnere dich an gestern: EAN-13 erkennt Zifferndreher zwischen Nachbarn nur dann, wenn die Differenz der beiden Ziffern **nicht 5** ist. Das ist ärgerlich – immerhin 11 % aller Zifferndreher bleiben unentdeckt.

Bei ISBN-10 ist das anders. Die Behauptung lautet:

ISBN-10 erkennt jeden Zifferndreher zwischen zwei beliebigen Positionen.

Beachte: Nicht nur Nachbarn, sondern **alle** Vertauschungen! Das ist deutlich stärker als EAN-13.



Bleistift-
Übung 2

Den Beweis mitdenken

- (a) Stell dir vor, in einer gültigen ISBN-10 werden die Ziffern an Position i und j vertauscht (sagen wir, die Ziffern sind a und b , mit $a \neq b$). Wie verändert sich die gewichtete Summe?

Tipp: Vorher steht in der Summe $a \cdot (11-i) + b \cdot (11-j)$. Nachher steht da $b \cdot (11-i) + a \cdot (11-j)$. Bilde die Differenz.

- (b) Du solltest auf eine Differenz der Form $(a - b) \cdot (j - i)$ gekommen sein. Damit der Tausch unentdeckt bleibt, müsste diese Differenz durch **11** teilbar sein. Warum ist das praktisch unmöglich?

Tipp: 11 ist eine **Primzahl**. Was heißt das für Produkte?

- (c) Was wäre, wenn wir Modulo 10 nehmen würden statt Modulo 11? Warum funktioniert das Argument dann nicht mehr?

ISBN-10-Validator



Python-
Einheit 1

Lege in Thonny eine neue Datei isbn10.py an:

```
def isbn10_pruefziffer(neun_ziffern):
    """Berechnet die ISBN-10-Prüfziffer für eine Liste von 9 Ziffern.
    Liefert eine Zahl zwischen 0 und 10 (10 bedeutet 'X')."""
    summe = 0
    for i, d in enumerate(neun_ziffern):
        gewicht = 10 - i
        summe += d * gewicht
    return (11 - summe % 11) % 11

def isbn10_ist_gueltig(isbn):
    """isbn ist ein String mit 10 Zeichen. Letztes Zeichen darf 'X' sein."""
    ziffern = []
    for c in isbn[:9]:
        ziffern.append(int(c))
    erwartet = isbn10_pruefziffer(ziffern)
    letztes = isbn[9]
    if letztes == 'X':
        return erwartet == 10
    else:
        return erwartet == int(letztes)

# Tests:
print(isbn10_ist_gueltig("0306406152")) # True
print(isbn10_ist_gueltig("344643500X")) # True (Prüfziffer ist X!)
print(isbn10_ist_gueltig("0306406153")) # False
```

Aufgabe 1.1 - Selbst Zifferndreher testen Schreibe (analog zu gestern) eine Funktion, die für eine gültige ISBN-10 **alle möglichen Vertauschungen zweier Positionen** durchprobiert und zählt, wie viele unentdeckt bleiben. Erwartung: **0**.

```
def teste_alle_zifferndreher_isbn10(isbn):
    nicht_erkant = []
    # TODO: Greta füllt aus
    # - zwei Schleifen über alle Positionspaare (i, j) mit i < j
    # - Ziffern tauschen
    # - prüfen, ob immer noch gültig
    return nicht_erkant
```

Tipp: Die letzte Position (Index 9) kann das Symbol 'X' sein. Sei beim Tauschen vorsichtig. Eine pragmatische Lösung: Tausche nur, wenn beide Positionen Ziffern (kein 'X') enthalten.

Bonus-Trick: ISBN-10 kann eine fehlende Ziffer rekonstruieren

Stell dir vor, ein Fettfleck verdeckt **eine** Ziffer einer ISBN-10. Du kannst die fehlende Ziffer ausrechnen!

Detektivarbeit

Hier ist eine ISBN-10, bei der die Ziffer an Position 5 (das ?) durch einen Tropfen Tee unleserlich geworden ist:

0-30?-40615-2

Finde die fehlende Ziffer.

Tipp: Du kennst die Summe modulo 11, die rauskommen muss (nämlich so, dass die Prüfziffer am Ende stimmt). Stelle eine Gleichung auf und löse nach ? auf.

Das ist Gretas erste echte Korrektur! Bisher hatten wir nur Erkennung. Mit ISBN-10 können wir tatsächlich eine fehlende Ziffer **rekonstruieren** – sofern wir wissen, *welche* Ziffer fehlt.

Achtung: Wenn wir nicht wissen, *welche* Ziffer fehlt (sondern nur, dass irgendwo ein Fehler ist), können wir nicht korrigieren. Das geht erst mit cleveren Verfahren wie Hamming-Codes (morgen!) oder Reed-Solomon.



Bleistift-
Übung 3

Block 2 · Luhn-Algorithmus - Kreditkarten und der Trick mit der Quersumme (≈ 30 min)

Hol mal eine Kreditkarte raus (oder schau dir eine im Internet an). Die 16-stellige Nummer trägt eine Prüfziffer am Ende, berechnet mit dem **Luhn-Algorithmus**, benannt nach Hans Peter Luhn (IBM, 1954).

Wie es funktioniert

1. Beginne **rechts** (bei der Prüfziffer) und gehe nach links.
2. **Verdopple jede zweite Ziffer**, beginnend mit der zweitletzten.
3. Wenn beim Verdoppeln eine zweistellige Zahl rauskommt (≥ 10), bilde die **Quersumme** (z. B. $16 \rightarrow 1 + 6 = 7$).
4. Bilde die Summe aller (manche verdoppelt, manche nicht) Ziffern.
5. Eine Nummer ist gültig, wenn die Summe durch **10** teilbar ist.

Beispiel: 4539 1488 0343 6467

Ziffern (von links): 4 5 3 9 1 4 8 8 0 3 4 3 6 4 6 7

Markieren wir die Ziffern, die verdoppelt werden (jede zweite von rechts, beginnend mit der zweitletzten):

Position:	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Ziffer:	4	5	3	9	1	4	8	8	0	3	4	3	6	4	6	7
Verdoppeln?	x		x		x		x		x		x		x		x	

Verdoppelte Ziffern ($\times 2$, dann Quersumme falls ≥ 10): $- 4 \cdot 2 = 8 - 3 \cdot 2 = 6 - 1 \cdot 2 = 2 - 8 \cdot 2 = 16 \rightarrow 1 + 6 = 7 - 0 \cdot 2 = 0 - 4 \cdot 2 = 8 - 6 \cdot 2 = 12 \rightarrow 1 + 2 = 3 - 6 \cdot 2 = 12 \rightarrow 3$

Nicht verdoppelt: 5, 9, 4, 8, 3, 3, 4, 7

Summe: $(8+6+2+7+0+8+3+3) + (5+9+4+8+3+3+4+7) = 37 + 43 = 80$

$80 \bmod 10 = 0 \rightarrow$ **gültig** ☐

Luhn von Hand

Bleistift-
Übung 4

- Prüfe die Nummer 4716 7300 5852 1908. Ist sie gültig?
- Erfinde eine 15-stellige Zahl deiner Wahl. Berechne die fehlende Prüfziffer (die 16. Ziffer), so dass die ganze Nummer Luhn-gültig wird.
- Eine Kassiererin tippt eine Nummer mit **einem Tippfehler** in einer einzigen Ziffer ein. Erkennt Luhn das? Probier es aus, indem du in der Nummer aus (a) eine Ziffer veränderst und nachrechnest.

Warum die Quersumme?

Hier liegt der eigentliche Witz von Luhn. Stell dir vor, wir würden **nicht** die Quersumme nehmen, sondern einfach die verdoppelte Zahl als Ganzes in die Summe schreiben. Dann wäre $8 \cdot 2 = 16$ und $3 \cdot 2 = 6$ – sehr unterschiedlich. Mit Quersumme wird daraus 7 und 6 – fast gleich.

Das ist *genau* gewollt: Mit Quersumme entspricht das Verdoppeln einer cleveren Abbildung von 0-9 auf 0-9, die einen wichtigen Zifferndreher-Fall mit abdeckt. Wir sparen uns hier die genaue Analyse, aber die Praxis-Bilanz ist:

Luhn erkennt **jeden Einzelfehler** und **die meisten Zifferndreher** zwischen Nachbarn.

(Genau genommen: alle Zifferndreher zwischen Nachbarn *außer* der Vertauschung $0 \leftrightarrow 9$. Greta darf das gerne in der Bonus-Aufgabe nachprüfen.)

Luhn-Validator

Python-
Einheit 2

```
def luhn_pruefen(number):
    """Prüft, ob 'number' (String aus Ziffern) Luhn-gültig ist."""
    summe = 0
    # Wir gehen von rechts nach links
    for i, c in enumerate(reversed(number)):
        d = int(c)
        if i % 2 == 1:
            # Jede zweite Ziffer von rechts (beginnend bei Index 1) verdoppeln
            d = d * 2
            if d >= 10:
                d = d - 9 # Trick: bei zweistelligem Ergebnis ist Quersumme = d - 9
        summe += d
    return summe % 10 == 0

# Tests:
print(luhn_pruefen("4539148803436467")) # True
print(luhn_pruefen("4539148803436468")) # False (letzte Ziffer falsch)
```

Mini-Trick erklärt: Wenn $d * 2$ zweistellig wird (also zwischen 10 und 18), dann ist die Quersumme genau $d * 2 - 9$. Probier es aus: $8 * 2 = 16$, Quersumme $1+6 = 7$, und $16 - 9 = 7$ □. Spart eine Zeile Code.

Aufgabe 2.1 - Welche Tippfehler erkennt Luhn nicht? Bau (analog zu Tag 1) einen Brute-Force-Test, der für eine Luhn-gültige Nummer **alle benachbarten Zifferndreher** durchprobiert. Welche bleiben unentdeckt?

Praxis-Einschub: Prüfziffer ≠ Sicherheit

Hier ist eine wichtige Frage: Wenn die Luhn-Prüfziffer öffentlich bekannt ist (jeder kann sie ausrechnen), warum schützt sie überhaupt irgendwas?

Die Antwort: Sie schützt **nicht vor Fälschung**, sondern vor **Tippfehlern**.

- Eine Luhn-gültige Kreditkartennummer ausdenken ist trivial (Greta hat das in Aufgabe 4(b) gemacht). Aber: Damit man tatsächlich Geld abbuchen könnte, müsste die ausgedachte Nummer auch in einer echten Datenbank existieren *und* Greta müsste das CVV (3-stellige Zahl auf der Rückseite) und das Ablaufdatum kennen.
- Was Luhn aber tut: Wenn ein Online-Shop eine Nummer empfängt, kann er **schon im Browser** prüfen, ob die 16 Ziffern überhaupt eine valide Form haben. Spart eine Anfrage an die Bank.

Diese Unterscheidung ist wichtig: - **Integritätsprüfung** (Hamming, Parität, ISBN, EAN, Luhn, später Reed-Solomon): „Sind die Daten unterwegs verfälscht worden?“ - **Sicherheit / Authentizität** (kryptographische Signaturen, Hashes, MACs): „Kommen die Daten wirklich von dem, von dem ich glaube, dass sie kommen? Hat jemand sie absichtlich manipuliert?“

Datamatrix-Codes haben Reed-Solomon-Korrektur drin – das ist **Integrität**, nicht Sicherheit. Ein Fälscher kann ohne Probleme einen gültigen Datamatrix mit erfundenem Inhalt drucken. Wenn ein Hersteller will, dass seine Codes

auch gegen Fälschung schützen, muss er **zusätzlich** etwas Kryptographisches einbauen (z. B. eine digitale Signatur, die im Inhalt steht).

Lass dieses Thema kurz wirken. Es ist eine der Sachen, die in der Praxis ständig verwechselt werden – auch von Erwachsenen mit Krawatte.

Block 3 · Hamming-Distanz - Die zentrale Idee, endlich (≈ 60 min)

Bis jetzt haben wir Verfahren *angeschaut* und ihre Eigenschaften (welche Fehler erkannt werden, welche nicht) ausprobiert. Jetzt kommt der Begriff, der erklärt, **warum** das alles funktioniert. Er ist erstaunlich einfach.

Definition

Die **Hamming-Distanz** zwischen zwei gleich langen Wörtern ist die Anzahl der Stellen, an denen sie sich unterscheiden.

```
1011010
1001110
^  ^ ^
    drei Unterschiede → Hamming-Distanz = 3
```

Die Hamming-Distanz funktioniert nicht nur für Bits, sondern für jedes Alphabet:

```
KATZE
KASSE
^ ^
    Hamming-Distanz = 2
```

Distanzen rechnen

Berechne:

Wort A	Wort B	Distanz
1011010	1011010	
0000	1111	
11001100	10101010	
HUND	RUND	
TANNE	KANNE	
0000000	1111111	



Bleistift-
Übung 5

Die Schlüsselidee: Mindestdistanz eines Codes

Ein **Code** ist einfach eine Menge gültiger Codewörter. Beispiel: Der Wiederholungscode (3,1) hat zwei Codewörter:

000 ← steht für 0
111 ← steht für 1

Die Hamming-Distanz zwischen 000 und 111 ist **3**. Das ist die **Mindestdistanz** dieses Codes (mit nur zwei Codewörtern gibt es eh nur eine Distanz).

Ein anderes Beispiel: der Paritätscode mit 3 Datenbits + 1 Paritätsbit. Codewörter:

0000 0011 0101 0110 1001 1010 1100 1111

Was ist die kleinste Distanz zwischen je zwei dieser acht Wörter?

Mindestdistanz finden

- Berechne die Distanzen zwischen einigen Paaren von Codewörtern aus der Paritätscode-Tabelle oben. Was ist die kleinste, die du findest?
- Vergleiche dein Ergebnis mit dem Wiederholungscode (000, 111). Welcher Code hat die größere Mindestdistanz?



Bleistift-
Übung 6

Die Magie der Mindestdistanz

Hier ist die zentrale Erkenntnis des Tages. Sie sieht so aus:

Wenn die **Mindestdistanz** eines Codes **d** ist, dann

- kann der Code **d – 1 Fehler erkennen**, und
- kann der Code **$\lfloor (d - 1) / 2 \rfloor$ Fehler korrigieren**.

(Das $\lfloor \cdot \rfloor$ heißt „abrunden“. Bei $d = 3$ ist $\lfloor (3-1)/2 \rfloor = \lfloor 1 \rfloor = 1$. Bei $d = 4$ wäre es ebenfalls 1. Bei $d = 5$ wären es 2.)

Warum das so ist - das Inselbild

Stell dir vor, jedes mögliche Bitwort einer Länge ist ein Punkt. Gültige Codewörter sind „Inseln“. Wenn unterwegs Bits kippen, springt das Wort von seiner Insel weg in Richtung Meer.

- Bei Mindestdistanz **3** sind alle Inseln mindestens 3 Schritte voneinander entfernt. Wenn 1 Bit kippt, landet das Wort 1 Schritt weg von seiner Insel – aber immer noch mindestens 2 Schritte weg von jeder *anderen* Insel. Der Empfänger weiß: „Die nächstgelegene Insel ist die richtige.“ → **1 Fehler korrigierbar**.

- Wenn 2 Bits kippen (Distanz 2 weg vom Original), könnte das Wort theoretisch genau in der Mitte zwischen zwei Inseln landen. Wir können dann zwar erkennen, dass etwas falsch ist (das Wort liegt nicht auf einer Insel), aber **nicht eindeutig korrigieren**. → **2 Fehler erkennbar**.

Daher: Distanz 3 = 1 korrigierbar oder 2 erkennbar (aber nicht beides gleichzeitig).



Bleistift-
Übung 7

Anwenden

Fülle die Tabelle aus:

Code	Mindestdistanz	Fehler erkennbar	Fehler korrigierbar
Paritätscode (3+1 Bit)			
Wiederholungscode (3,1)			
Wiederholungscode (5,1)			
Wiederholungscode (7,1)			

Was beobachtest du beim Vergleich der Wiederholungscode? Wie wächst die Korrekturkraft mit der Länge?



Python-
Einheit 3

Hamming-Distanz selbst berechnen

```
def hamming_distanz(a, b):
    """Liefert die Hamming-Distanz zwischen zwei gleich langen Sequenzen."""
    if len(a) != len(b):
        raise ValueError("Wörter müssen gleich lang sein")
    distanz = 0
    for x, y in zip(a, b):
        if x != y:
            distanz += 1
    return distanz

# Tests:
print(hamming_distanz("KATZE", "KASSE"))      # 2
print(hamming_distanz([0,0,0], [1,1,1]))      # 3
print(hamming_distanz("1011010", "1001110"))  # 3
```

Aufgabe 3.1 - Mindestdistanz eines Codes Schreibe eine Funktion `mindestdistanz(codewoerter)`, die eine Liste von Codewörtern bekommt und die kleinste Hamming-Distanz zwischen je zwei davon zurückliefert.

```
def mindestdistanz(codewoerter):
    # TODO: Greta füllt aus
    # Tipp: itertools.combinations(codewoerter, 2) liefert alle Paare
    pass
```

```
# Test mit dem Paritätscode (3+1 Bit):
paritaet_code = ["0000", "0011", "0101", "0110",
                 "1001", "1010", "1100", "1111"]
print(mindestdistanz(paritaet_code)) # erwartet: 2

# Test mit dem Wiederholungscode (3,1):
wdh_code = ["000", "111"]
print(mindestdistanz(wdh_code))      # erwartet: 3
```

Aufgabe 3.2 - Fehlerkorrektur per Brute Force Greift Distanz wirklich?
Lass uns das überprüfen.

```
import itertools

def naechstes_codewort(empfangen, codewoerter):
    """Findet das Codewort mit kleinster Distanz zum empfangenen Wort."""
    bestes = None
    beste_distanz = float('inf')
    for c in codewoerter:
        d = hamming_distanz(empfangen, c)
        if d < beste_distanz:
            beste_distanz = d
            bestes = c
    return bestes

# Wiederholungscode (3,1) auf 1-Bit-Fehlerkorrektur testen:
wdh_code = ["000", "111"]

richtig = 0
gesamt = 0
for original in wdh_code:
    for pos in range(3):
        empfangen = list(original)
        empfangen[pos] = '1' if empfangen[pos] == '0' else '0'
        empfangen = ''.join(empfangen)
        rekonstruiert = naechstes_codewort(empfangen, wdh_code)
        gesamt += 1
        if rekonstruiert == original:
            richtig += 1
print(f"{richtig}/{gesamt} Einzelfehler korrigiert")
```

Lass den Code laufen. Erwartung: 6/6, also 100 %.

Aufgabe 3.3 - Was passiert bei 2 Fehlern? Erweitere den Test, sodass **2 Bits** gleichzeitig kippen. Wie viel Prozent werden noch richtig korrigiert?

Tipp: `itertools.combinations(range(3), 2)` liefert alle Paare (0,1), (0,2), (1,2).

Du wirst sehen: deutlich weniger – bei Distanz 3 ist 1 Fehler die garantierte Korrekturkraft, mehr nicht.

Block 4 · Reflexion und Ausblick (≈ 15 min)

Was wir heute gelernt haben

- **ISBN-10** ist EAN-13 in schlauer: Modulo 11 statt 10. Dadurch werden *alle* Zifferndreher (nicht nur benachbarte) erkannt – um den Preis, dass manchmal das Symbol „X“ als Prüfziffer nötig ist.
- ISBN-10 kann sogar **eine fehlende Ziffer rekonstruieren** – sofern man weiß, welche Position fehlt. Das ist erste echte Korrektur.
- **Luhn** für Kreditkarten ist clever konstruiert (Verdoppeln + Quersumme), aber nur eine Integritätsprüfung. Vor Fälschung schützt sie nicht – das wäre ein anderes Werkzeug (Kryptographie).
- Die **Hamming-Distanz** ist *das* zentrale Maß für Codes: Je größer die Mindestdistanz, desto mehr Fehler kann ein Code erkennen oder korrigieren.
- Konkret: Mindestdistanz $d \rightarrow$ bis zu $d-1$ **Fehler erkennbar**, bis zu $\lfloor (d-1)/2 \rfloor$ **korrigierbar**.

Knobelaufgabe als Cliffhanger für Tag 3

Wir haben gesehen: Wiederholungscode (3,1) korrigiert 1 Fehler, kostet aber 67 % Redundanz. Wiederholungscode (5,1) korrigiert 2 Fehler, kostet 80 % Redundanz. Das ist ziemlich teuer.

Frage: Du hast 4 Datenbits. Wie viele Prüfbits brauchst du **mindestens**, damit du jeden Einzel-Bitfehler korrigieren kannst?

Tipp 1: Es gibt $2^4 = 16$ mögliche Datenwörter. Jedes davon plus seine 1-Bit-Nachbarn muss eindeutig zu *seinem* Codewort gehören.

Tipp 2: Wie viele 1-Bit-Nachbarn hat ein Wort der Länge 7?

Versuche, die Antwort vor Tag 3 zu finden. Morgen lösen wir es gemeinsam – und bauen den ersten echten fehlerkorrigierenden Code: den **(7,4)-Hamming-Code**.

Vorschau Tag 3

Morgen lernst du den Mann kennen, der dieses Verfahren erfunden hat (Richard Hamming, ein leicht genervter Mathematiker an einem Lochkarten-Wochenende), und du baust seinen Code selbst – mit Bleistift, drei überlappenden Kreisen und Python. Am Ende kannst du **4 Datenbits in 7 Bits codieren**, und dein Decoder zeigt dir nicht nur, *dass* ein Fehler passiert ist, sondern **an welcher Position** – und korrigiert ihn automatisch.

Lösungen (erst nach eigenem Versuch ansehen!)

Bleistiftübung 1 - ISBN-10

(b) 0306406152:

$$\begin{aligned}
 &0 \cdot 10 + 3 \cdot 9 + 0 \cdot 8 + 6 \cdot 7 + 4 \cdot 6 + 0 \cdot 5 + 6 \cdot 4 + 1 \cdot 3 + 5 \cdot 2 \\
 &= 0 + 27 + 0 + 42 + 24 + 0 + 24 + 3 + 10 = 130 \\
 &130 \bmod 11 = 9 \quad (\text{denn } 130 = 11 \cdot 11 + 9) \\
 &11 - 9 = 2
 \end{aligned}$$

Prüfziffer = **2**, also 0-306-40615-2. Das ist „Gödel, Escher, Bach“ von Douglas Hofstadter – ein Klassiker.

(c) 097522980:

$$\begin{aligned}
 &0 \cdot 10 + 9 \cdot 9 + 7 \cdot 8 + 5 \cdot 7 + 2 \cdot 6 + 2 \cdot 5 + 9 \cdot 4 + 8 \cdot 3 + 0 \cdot 2 \\
 &= 0 + 81 + 56 + 35 + 12 + 10 + 36 + 24 + 0 = 254 \\
 &254 \bmod 11 = 1 \quad (\text{denn } 254 = 23 \cdot 11 + 1) \\
 &11 - 1 = 10
 \end{aligned}$$

Prüfziffer = **10**, geschrieben als **X**. Vollständige ISBN: 097522980X.

Bleistiftübung 2 - Warum Modulo 11

(a) Differenz: $(b - a) \cdot (11 - i) + (a - b) \cdot (11 - j) = (a - b) \cdot ((11 - j) - (11 - i)) = (a - b) \cdot (i - j)$.

(b) Damit der Tausch unentdeckt bleibt, müsste $(a - b) \cdot (i - j) \equiv 0 \pmod{11}$. Da 11 **prim** ist, gilt: ein Produkt ist nur dann durch 11 teilbar, wenn einer der Faktoren durch 11 teilbar ist. Aber $a - b$ liegt zwischen -9 und 9 (und ist nicht 0, weil $a \neq b$), und $i - j$ liegt zwischen -8 und 8 (und ist nicht 0, weil verschiedene Positionen). Beide Faktoren sind also **niemals durch 11 teilbar** – der Tausch wird immer erkannt!

(c) Bei Modulo 10 funktioniert das Argument nicht, weil $10 = 2 \cdot 5$ (keine Primzahl). Beispiel: $a - b = 5$ und $i - j = 2$ ergibt $10 \rightarrow$ Tausch unentdeckt. Genau das Phänomen aus Tag 1.

Bleistiftübung 3 - Detektivarbeit

ISBN: 0-30?-40615-2 mit unbekannter Ziffer x an Position 4.

Gewichtete Summe inklusive Prüfziffer 2:

$$\begin{aligned}
 &0 \cdot 10 + 3 \cdot 9 + 0 \cdot 8 + x \cdot 7 + 4 \cdot 6 + 0 \cdot 5 + 6 \cdot 4 + 1 \cdot 3 + 5 \cdot 2 + 2 \cdot 1 \\
 &= 0 + 27 + 0 + 7x + 24 + 0 + 24 + 3 + 10 + 2 \\
 &= 90 + 7x
 \end{aligned}$$

Damit die ISBN gültig ist, muss diese Summe durch 11 teilbar sein:

$$90 + 7x \equiv 0 \pmod{11}$$

$$90 \equiv 2 \pmod{11}$$

$$\text{also: } 2 + 7x \equiv 0 \pmod{11}$$

$$7x \equiv -2 \equiv 9 \pmod{11}$$

Probiere $x = 0, 1, \dots, 9$: $x = 6$ ergibt $7 \cdot 6 = 42 \equiv 9 \pmod{11}$ $\square$.

Die fehlende Ziffer ist **6**, die ISBN lautet 0-306-40615-2.

Aufgabe 1.1 - Lösung

```
def teste_alle_zifferndreher_isbn10(isbn):
    nicht_erkant = []
    zeichen = list(isbn)
    for i in range(10):
        for j in range(i+1, 10):
            # Pragmatisch: nur tauschen, wenn beide Ziffern sind
            if zeichen[i] == 'X' or zeichen[j] == 'X':
                continue
            if zeichen[i] == zeichen[j]:
                continue
            getauscht = zeichen.copy()
            getauscht[i], getauscht[j] = getauscht[j], getauscht[i]
            if isbn10_ist_gueltig(''.join(getauscht)):
                nicht_erkant.append((i, j, ''.join(getauscht)))
    return nicht_erkant

print(teste_alle_zifferndreher_isbn10("0306406152"))
# -> [] (keine unentdeckten Vertauschungen!)
```

Bleistiftübung 4 - Luhn

(a) 4716 7300 5852 1908: Von rechts nach links Position-Index 0, 1, 2, ...

8 (× nicht)

$$0 \rightarrow 0 \cdot 2 = 0$$

9 (× nicht)

$$1 \rightarrow 1 \cdot 2 = 2$$

2 (× nicht)

$$5 \rightarrow 5 \cdot 2 = 10 \rightarrow 1$$

8 (× nicht)

$$5 \rightarrow 5 \cdot 2 = 10 \rightarrow 1$$

0 (× nicht)

$$0 \rightarrow 0 \cdot 2 = 0$$

3 (× nicht)

$$7 \rightarrow 7 \cdot 2 = 14 \rightarrow 5$$

6 (× nicht)

$$1 \rightarrow 1 \cdot 2 = 2$$

7 (× nicht)
 4 → 4 · 2 = 8

Summe = 8+0+9+2+2+1+8+1+0+0+3+5+6+2+7+8 = 62
 62 mod 10 = 2 ≠ 0 → ungültig

Die Nummer ist **nicht** gültig.

- (c) Die meisten Einzelfehler werden erkannt. Tatsächlich: Luhn erkennt **alle** Einzelfehler in einer einzelnen Ziffer.

Aufgabe 2.1 - Welche Tippfehler bleiben unentdeckt?

```
def teste_zifferndreher_luhn(nummer):
    nicht_erkant = []
    zeichen = list(nummer)
    for i in range(len(zeichen) - 1):
        if zeichen[i] == zeichen[i+1]:
            continue
        getauscht = zeichen.copy()
        getauscht[i], getauscht[i+1] = getauscht[i+1], getauscht[i]
        if luhn_pruefen(''.join(getauscht)):
            nicht_erkant.append((i, ''.join(getauscht)))
    return nicht_erkant
```

Mit verschiedenen Luhn-gültigen Nummern testen.

Ergebnis: Die einzige Vertauschung benachbarter Ziffern, die Luhn **nicht** erkennt, ist 09 ↔ 90. Alle anderen werden erkannt.

Bleistiftübung 5 - Distanzen

A	B	Distanz
1011010	1011010	0
0000	1111	4
11001100	10101010	4
HUND	RUND	1
TANNE	KANNE	1
0000000	1111111	7

Bleistiftübung 6 - Mindestdistanz

- (a) Schauen wir z. B. 0000 vs 0011: Distanz 2. Oder 0011 vs 0101: Distanz 2. Tatsächlich ist die Mindestdistanz beim Paritätscode immer **2** (jedes Codewort unterscheidet sich vom nächstgelegenen genau in 2 Bits, weil jede Veränderung von 1 Bit die Parität kippen würde).

- (b) Wiederholungscode (000, 111) hat Minstdistanz **3**, also größer als der Paritätscode (2).

Bleistiftübung 7 - Tabelle

Code	Minstdistanz	Erkennbar	Korrigierbar
Paritätscode (3+1)	2	1	0
Wiederholungscode (3,1)	3	2	1
Wiederholungscode (5,1)	5	4	2
Wiederholungscode (7,1)	7	6	3

Beobachtung: Bei Wiederholungscode der Länge **n** (n ungerade) ist die Minstdistanz n, also können $(n-1)/2$ Fehler korrigiert werden. Sehr robust, aber sehr ineffizient.

Aufgabe 3.1 - Lösung

```
import itertools

def minstdistanz(codewoerter):
    minimum = float('inf')
    for a, b in itertools.combinations(codewoerter, 2):
        d = hamming_distanz(a, b)
        if d < minimum:
            minimum = d
    return minimum
```

Aufgabe 3.3 - Doppelfehler beim Wiederholungscode

```
import itertools

richtig = 0
gesamt = 0
for original in wdh_code:
    for pos1, pos2 in itertools.combinations(range(3), 2):
        empfangen = list(original)
        for pos in (pos1, pos2):
            empfangen[pos] = '1' if empfangen[pos] == '0' else '0'
        empfangen = ''.join(empfangen)
        rekonstruiert = naechstes_codewort(empfangen, wdh_code)
        gesamt += 1
        if rekonstruiert == original:
            richtig += 1
print(f"{richtig}/{gesamt} Doppelfehler korrigiert")
# → 0/6 (0 %) Bei 2 Fehlern liegt das Wort näher am falschen Codewort.
```

Das ist genau, was die Theorie vorhersagt: Distanz 3 → korrigierbar nur 1 Fehler.

Cliffhanger-Knobelaufgabe

Bei 4 Datenbits gibt es $2^4 = 16$ mögliche Datenwörter. Jedes davon, kodiert in **n** Bits, hat **n** mögliche 1-Bit-Nachbarn. Damit alle 16 Codewörter und ihre n Nachbarn nicht überlappen, brauchen wir mindestens $16 \cdot (n+1) \leq 2^n$.

- $n = 5$: $16 \cdot 6 = 96$, aber $2^5 = 32 \rightarrow$ reicht nicht.
- $n = 6$: $16 \cdot 7 = 112$, aber $2^6 = 64 \rightarrow$ reicht nicht.
- $n = 7$: $16 \cdot 8 = 128$, und $2^7 = 128 \rightarrow$ **passt genau**.

Antwort: **7 Bits** (also 3 Prüfbits zusätzlich zu den 4 Datenbits). Das ist der berühmte **(7,4)-Hamming-Code** – und das Erstaunliche: Die Schranke wird *exakt* getroffen, kein Bit ist zu viel.